

画像処理演習書：学び直し

2026.6.25

2026 年度

松井 進

目次

1	NumPy ライブラリ	4
1.1	配列の定義	4
1.2	配列の次元数、形、サイズ	5
1.3	配列の要素の参照	5
1.4	配列の四則演算	6
1.5	条件に応じた処理	8
1.6	平均値、合計値、最大値、最小値	9
1.7	データ型	11
1.8	スライスによる要素の選択	12
2	画像処理の基本	16
2.1	画像の読み込みと表示	16
2.2	画像サイズの取得	17
2.3	画像の保存	18
2.4	濃淡画像化	18
2.5	二値化	19
2.6	画像オブジェクトの定義	20
2.7	図形の描画	21
2.7.1	円の描画	21
2.7.2	矩形、線分の描画	22
3	マウス入力の処理	22
4	画像の平滑化	25
4.1	平滑化処理	25
4.2	移動平均フィルタ	26
4.2.1	ガウシアンフィルタ	27
5	色認識	28
5.1	HSV 色空間	28
5.2	色によるマスク画像の作成	30

6	輪郭抽出	31
---	------	----

授業目次

第 9 回目授業	2026.6.26	4
----------	-----------	---

問目次

問【1】	【配列の定義 (1)】	4
問【2】	【配列の定義 (2)】	5
問【3】	【配列の要素の参照】	6
問【4】	【配列の四則演算 (1)】	7
問【5】	【配列の四則演算 (2)】	7
問【6】	【配列の四則演算 (3)】	7
問【7】	【条件に応じた処理 (1)】	8
問【8】	【条件に応じた処理 (2)】	9
問【9】	【平均値、合計値、最大値、最小値】	10
問【10】	【スライス (1)】	14
問【11】	【スライス (2)】	14
問【12】	【スライス (3)】	14
問【13】	【スライス (4)】	14
問【14】	【スライス (5)】	15
問【15】	【スライス (6)】	15
問【16】	【スライス (7)】	16
問【17】	【jpeg 画像の表示】	17
問【18】	【画像のサイズ】	17
問【19】	【濃淡画像化 (1)】	18
問【20】	【濃淡画像化 (2)】	19
問【21】	【二値化】	20
問【22】	【画像オブジェクトの定義 (1)】	20
問【23】	【画像オブジェクトの定義 (2)】	21
問【24】	【円の描画】	21
問【25】	【マウスイベント】	25

問【26】	【移動平均フィルタ】	26
問【27】	【ガウシアンフィルタ】	28
問【28】	【HSV 値】	30
問【29】	【色認識によるマスク画像の作成】	30
問【30】	【輪郭抽出】	33
問【31】	【外接矩形の描画】	34

関数目次

np.array	4
np.zeros	4
np.ones	4
np.full	4
np.where	8
np.mean	9
cv2.imread	16
cv2.imwrite	18
cv2.cvtColor	18
cv2.cvtColor	19
cv2.circle	21
cv2.rectangle	22
cv2.line	22
cv2.setMouseCallback	22
onMouse	22
cv2.blur	26
cv2.GaussianBlur	27
cv2.inRange	30
cv2.findContours	31
cv2.drawContours	32
cv2.countourArea	33
cv2.arcLength	33
cv2.boundingRect	33
cv2.minEnclosingCircle	33

1 NUMPY ライブラリ

第 9 回目授業 (2026.6.26)

1 NumPy ライブラリ

1.1 配列の定義

NumPy では ndarray という多次元配列を使うことができる。ndarray は Python の list に似ているが、より高速処理が可能である。

1 次元配列の定義

```
a = np.array([1,2,3,4,5])
```

2 次元配列の定義 (以下では 3x5 の配列を定義)

```
a = np.array([[1,2,3,4,5],
              [6,7,8,9,10],
              [11,12,13,14,15]])
```

要素がすべて 0 の 2x3 配列の定義

```
a = np.zeros((2,3))
```

要素がすべて 1 の 4x5 配列の定義

```
a = np.ones((4,5))
```

要素がすべて 9 の 3x2 配列の定義

```
a = np.full((4,5),9)
```

問【1】【配列の定義 (1)】

要素が 1,2,3,4,5 からなる python のリストと Numpy 配列を定義し、プリントアウトするプログラム ex1.py を作成せよ。

問【2】【配列の定義 (2)】

要素が全て 10 の 4x5 の 2 次元 Numpy 配列を定義して、プリントアウトするプログラム `ex2py` を作成せよ。

1.2 配列の次元数、形、サイズ

`ndarray` には以下の属性がある。

```
ndim: 次元数
shape: 形 (各次元のサイズ)
size: 全要素数
```

```
1 import numpy as np
2
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     print(a.ndim)          # 2 と表示される
9     print(a.shape[0])      # 3 と表示される
10    print(a.shape[1])      # 5 と表示される
11    print(a.size)          # 3 x 5 = 15 と表示される
12
13 if __name__ == '__main__':
14     main()
```

1.3 配列の要素の参照

Python のリストでは多次元リストの要素の参照は `list[i][j]` であったが、`ndarray` では `ndarray[i,j]` である。

```
1 import numpy as np
2
```

```
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     print(a[0,0])          # 1 と表示される
9     print(a[1,0])          # 6 と表示される
10    print(a[2,4])           # 15 と表示される
11
12 if __name__ == '__main__':
13     main()
```

問【3】【配列の要素の参照】

要素が [1,2,3,4,5],[6,7,8,9,10] からな python の 2 次元リストと Numpy の 2 次元配列を定義し、インデックスを用いて要素を取り出してプリントアウトするプログラム ex3.py を作成せよ。

1.4 配列の四則演算

同じ形の ndarray の要素間の四則演算は配列名間の演算として簡単に記述できる。また、要素ごとの演算を行う関数が用意されている。

```
1 import numpy as np
2
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     b = np.array([[21,22,23,24,25],
9                   [26,27,28,29,30],
10                  [31,32,33,34,35]])
11
```

```
12 print(a+b)           # a の要素と対応する b の要素の和
13 print(a-b)           # a の要素と対応する b の要素の差
14 print(a*b)           # a の要素と対応する b の要素の積
15 print(a/b)           # a の要素と対応する b の要素の商
16 print(3*a)           # a の各要素を 3 倍
17 print(np.power(a,2))  # a の各要素の 2 乗
18 print(np.sqrt(a))     # a の各要素の平方根
19
20 if __name__ == '__main__':
21     main()
```

問【4】【配列の四則演算（1）】

要素が [1,2,3,4,5] の python のリストと Numpy の配列を定義し、各要素の値を 2 倍にするプログラム ex4.py を作成せよ

問【5】【配列の四則演算（2）】

要素が [1,2,3,4,5],[6,7,8,9,10] からなる python の 2 次元リストと Numpy の 2 次元配列を定義し、各要素の値に 10 を加えるプログラム ex5.py を作成せよ

問【6】【配列の四則演算（3）】

要素が [1,2,3,4,5],[6,7,8,9,10] 及び [21,22,23,24,25],[26,27,28,29,30] からなる 2 つの python の 2 次元リストと 2 つの Numpy の 2 次元配列を定義し、2 つの要素の和を要素とする新しいリスト及び Numpy 配列を作成するプログラム ex6.py を作成せよ

1.5 条件に応じた処理

`np.where()` を使うと、`ndarray` の条件を満たす要素を置き換えたり、特定の処理を行うことができる。以下に例を示す。

```
1 import numpy as np
2
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     b = np.where(a > 10, 255, a)
9     # 要素が 10 より大きい場合には 255 に置換、そうでない場合は変更しない
10    print(b)
11    # [[ 1  2  3  4  5]
12    #   [ 6  7  8  9 10]
13    #   [255 255 255 255 255]]
14
15    c = np.where(a > 10, 255, 0)
16    # 要素が 10 より大きい場合には 255 に置換、そうでない場合は 0 に置換
17    print(c)
18    # [[ 0  0  0  0  0]
19    #   [ 0  0  0  0  0]
20    #   [255 255 255 255 255]]
21
22    d = np.where(a < 10, a*2, a)
23    # 要素が 10 より大きい場合には要素の値を 2 倍する
24    print(d)
25    # [[ 2  4  6  8 10]
26    #   [12 14 16 18 10]
27    #   [11 12 13 14 15]]
28
29 if __name__ == '__main__':
30     main()
```

問【7】【条件に応じた処理（1）】

要素が [1,2,3,4,5,6,7,8,9,10] の python のリストと Numpy 配列を定義し、要素値が 5 以上の場合にはそこに要素の値を 10 にするプログラム ex7.py を作成せよ

問【8】【条件に応じた処理（2）】

要素が [1,2,3,4,5],[6,7,8,9,10],[11,12,13,14,15] の python のリストと Numpy 配列を定義し、要素値が 8 以上の場合にはそこに要素の値 255 にし、8 未満の場合には 0 するプログラム ex8.py を作成せよ

1.6 平均値、合計値、最大値、最小値

NumPy 配列の平均は np.mean() を用いて、以下のように求めることができる。

```
1 import numpy as np
2
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     print(np.mean(a))          # すべての要素の平均
9     # 8.0
10    print(np.mean(a,axis=0))    # 列の平均 a[i,j] の i に関する平均
11    # [ 6.  7.  8.  9. 10.]
12    print(np.mean(a,axis=1))    # 行の平均 a[i,j] の j に関する平均
13    # [ 3.  8. 13.]
14
15 if __name__ == '__main__':
16     main()
```

ここで、`axis=0` は多次元配列の一つ目のインデックス、`axis=1` は二つ目のインデックスを意味する。その他、合計を求める `np.sum()`、最大値を求める `np.max()`、最小値を求める `np.min()` などの関数が用意されている

問【9】【平均値、合計値、最大値、最小値】

要素が `[[[1,2,3,4,5],[6,7,8,9,10]],`
`[[11,12,13,14,15],[16,17,18,19,20]],`
`[[21,22,23,24,25],[26,27,28,29,30]]]`

の 3 次元配列の全要素の平均値、次元ごとの平均値、全要素の合計値、次元ごとの合計値、全要素の最大値、次元ごとの最大値、全要素の最小値、次元ごとの最小値を求めるプログラム `ex9.py` を作成せよ

1.7 データ型

ndarray の要素はすべて同じデータ型をとる。表 1.1 に ndarray の主なデータ型を示す。

表 1.1 ndarray の主なデータ型

データ型	説明
int8	符号あり 8 ビット整数型
int16	符号あり 16 ビット整数型
int32	符号あり 32 ビット整数型
int64	符号あり 64 ビット整数型
uint8	符号なし 8 ビット整数型
uint16	符号なし 16 ビット整数型
uint32	符号なし 32 ビット整数型
uint64	符号なし 64 ビット整数型
float16	半精度浮動小数点型
float32	単精度浮動小数点型
float64	倍精度浮動小数点型

ndarray のデータ型は dtype という属性に保存される。

```

1 import numpy as np
2
3 def main():
4     a = np.array([[1,2,3,4,5],
5                   [6,7,8,9,10],
6                   [11,12,13,14,15]])
7
8     b = np.array([[1.0,2.0,3.0,4.0,5.0],
9                   [6.0,7.0,8.0,9.0,10.0],
10                  [11.0,12.0,13.0,14.0,15.0]])
11
12     print(a.dtype)      # int64 と表示
13     print(b.dtype)      # float64 と表示
14
```

```
15 if __name__ == '__main__':  
16     main()
```

上記のように、通常の設定ではデータ型は int64 あるいは float64 となる。

特定のデータ型を指定して配列を宣言するのは `dtype = np.float64` などとする。また、配列のデータ型を変更するには `astype()` を使用する。

```
1 import numpy as np  
2  
3 def main():  
4     a = np.array([[1,2,3,4,5],  
5                  [6,7,8,9,10],  
6                  [11,12,13,14,15]])  
7  
8     # データ型を指定して配列を定義  
9     b = np.array([[1,2,3,4,5],  
10                 [6,7,8,9,10],  
11                 [11,12,13,14,15]], dtype=np.float64)  
12  
13     c = a.astype(np.int8) # 配列のデータ型を変更  
14  
15     print(a.dtype)        # int64 と表示される  
16     print(b.dtype)        # float64 と表示される  
17     print(c.dtype)        # int8 と表示される  
18  
19 if __name__ == '__main__':  
20     main()
```

1.8 スライスによる要素の選択

ndarray においても、Python のリストと同様に、スライスにより配列の一部を取得できる。記述は `[start:stop:step]` である。以下に例を示す。

```
1 import numpy as np  
2  
3 def main():  
4  
5     a = np.array([1,2,3,4,5])
```

```
6
7 print(a[:2]);
8     # [1,2]
9
10 print(a[1:3]);
11     # [2,3]
12
13 print(a[3:]);
14     # [4,5]
15
16 print(a[:,2]);
17     # [4,5]
18
19 b = np.array([[1,2,3,4,5],
20               [6,7,8,9,10],
21               [11,12,13,14,15],
22               [16,17,18,19,20],
23               [21,22,23,24,25],
24               [26,27,28,29,30]])
25
26 print(b[2:4,:])
27     # [[11 12 13 14 15]
28     #   [16 17 18 19 20]]
29
30 print(b[2:4,2:4])
31     # [[13 14]
32     #   [18 19]]
33
34 print(b[:,2:4])
35     # [[ 3  4]
36     #   [ 8  9]
37     #   [13 14]
38     #   [18 19]
39     #   [23 24]
40     #   [28 29]]
41
42 print(b[:,::2])
43     # [[ 1  3  5]
44     #   [ 6  8 10]
45     #   [11 13 15]
```

```
46     # [16 18 20]
47     # [21 23 25]
48     # [26 28 30]]
49
50     print(b[:, ::2, ::2])
51     # [[ 1  3  5]
52        # [11 13 15]
53        # [21 23 25]]
54
55 if __name__ == '__main__':
56     main()
```

問【10】【スライス（1）】

要素が [1,2,3,4,5,6,7,8,9,10] の Numpy 配列の最初の 3 要素のみからなる新たな配列を求めるプログラム ex10.py を作成せよ

問【11】【スライス（2）】

要素が [1,2,3,4,5,6,7,8,9,10] の Numpy 配列の最初の 4 番目の要素から 8 番目の要素からなる新たな配列を求めるプログラム ex11.py を作成せよ

問【12】【スライス（3）】

要素が [1,2,3,4,5,6,7,8,9,10] の Numpy 配列の最初の 1 番目、3 番目、・・・と一つおきに要素からなる新たな配列を求めるプログラム ex12.py を作成せよ

問【13】【スライス（4）】

要素が `[1,2,3,4,5,6,7,8,9,10]` の Numpy 配列の要素順を逆順にしたからなる新たな配列を求めるプログラム `ex13.py` を作成せよ

問【14】【スライス（5）】

要素が `[[1,2,3,4,5],`

`[6,7,8,9,10],`

`[11,12,13,14,15],`

`[16,17,18,19,20],`

`[21,22,23,24,25],`

`[26,27,28,29,30]]`

の Numpy 配列の 1 行おきからなる新たな配列を求めるプログラム `ex14.py` を作成せよ

問【15】【スライス（6）】

要素が `[[1,2,3,4,5],`

`[6,7,8,9,10],`

`[11,12,13,14,15],`

`[16,17,18,19,20],`

`[21,22,23,24,25],`

`[26,27,28,29,30]]`

の Numpy 配列の 1 列おきからなる新たな配列を求めるプログラム `ex15.py` を作成せよ

問【16】【スライス（7）】

要素が [[1,2,3,4,5],
[6,7,8,9,10],
[11,12,13,14,15],
[16,17,18,19,20],
[21,22,23,24,25],
[26,27,28,29,30]]

の Numpy 配列の 2 行目から 3 行目、1 列目から 4 列目からなる新たな配列を求めるプログラム ex16.py を作成せよ

2 画像処理の基本

2.1 画像の読み込みと表示

下記プログラムは画像を読み込みウィンドに表示するプログラムである。

```
1 import numpy as np
2 import cv2
3
4 def main():
5     img = cv2.imread("./ktech.jpg")
6
7     cv2.imshow("Image", img)
8
9     cv2.waitKey(0)
10    cv2.destroyAllWindows()
11
12 if __name__ == '__main__':
13     main()
```

以下プログラムを説明する。

- 1 行目で NumPy を、2 行目で OpenCV を import している。
- 5 行目で jpeg 画像を読み込み、img オブジェクト（NumPy 配列 ndarray）に代入

している。

- 7 行目で読み込んだ画像をウィンドに表示している。
- 9 行目はなにかキーが押下されるのを待つ命令である（表示した画像にフォーカスがあたっている必要がある）。
- 10 行目で全ウィンドを閉じている。

問【17】【jpeg 画像の表示】

kteck.jpg ファイルを読み込み、表示するプログラム ex17.py を作成せよ

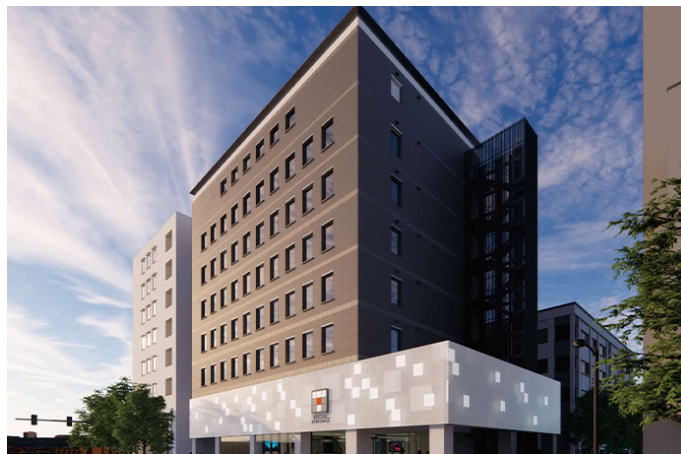


図 2.1 画像の表示

2.2 画像サイズの取得

cv2.imread を画像を読み込むと、画像は NumPy の ndarray 配列である変数 img に読み込んでいる。読み込んだ画像はカラー画像であり、img オブジェクトは 3 次元の ndarray である。

問【18】【画像のサイズ】

Numpy の属性を参考に、読み込んだ画像の `height` と `width` を表示するプログラム `ex18.py` を作成せよ

2.3 画像の保存

画像の保存は `cv2.imwrite()` で行うことができる。

以下に画像を保存するプログラムを示す。

```
1 import numpy as np
2 import cv2
3
4 def main():
5     img = cv2.imread("../figure/ktech.jpg")
6     cv2.imshow("Image", img)
7
8     cv2.imwrite("./ktech2.jpg",img)
9
10    cv2.waitKey(0)
11    cv2.destroyAllWindows()
12
13 if __name__ == '__main__':
14     main()
```

2.4 濃淡画像化

濃淡画像化（グレイ化）は `cv2.cvtColor()` 関数で行うことができる

```
1 # 濃淡画像
2 img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

問【19】【濃淡画像化（1）】

ktech.jpg をグレースケール化して表示するプログラム ex19.py を作成せよ



図 2.2 濃淡画像の表示

問【20】【濃淡画像化（2）】

濃淡画像化前の画像の次元数及び濃淡画像化後の画像の次元数を表示するプログラム ex20.py を作成せよ

2.5 二値化

二値化はグレイ画像を入力し、`cv2.threshold()` 関数で行うことができる

```
1 # 二値化
2 th, img_bin = cv2.threshold(img_gray, thresh, maxval, cv2.
3     THRESH_BINARY)
```

```
4 thresh: 閾値、これより大きい場合には maxval、小さい場合には 0 に設定する  
5     通常は 127 とする  
6 maxval 閾値を超えた場合に設定される値、通常は 255 とする)
```

問【21】【二値化】

ktech.jpg をグレースケール化し、二値化して表示するプログラム ex21.py を作成せよ



図 2.3 二値化画像の表示

2.6 画像オブジェクトの定義

これまでは画像ファイルを読み込んで画像オブジェクトとしていたが、画像オブジェクトを定義することができる。カラーの画像オブジェクトは高さ、幅、BGR の 3 次元 Numpy 配列である。3 次元目の順番は BGR の順であることに注意すること。値は 0 から 255 であり、データ型は np.uint8 である。

問【22】【画像オブジェクトの定義（1）】

高さ 600、幅 800、色が白（RGB で (255,255,255)）のカラー画像オブジェクトを定義し、表示するプログラム `ex22.py` を作成せよ。

問【23】【画像オブジェクトの定義（2）】

高さ 600、幅 800、色が黄色（RGB で (255,255,0)）の画像オブジェクトを定義し、表示するプログラム `ex23.py` を作成せよ。

2.7 図形の描画

2.7.1 円の描画

円の描画は `cv2.circle()` 関数を使用する。

```
# 円の描画
cv2.circle(img, (x, y), r, (b, g, r), weight)
# パラメータ：(x,y) 中心座標、f: 半径、線の色（BGR）、
# weight: 線の太さ（負数の場合は塗りつぶし）
# 中心座標は整数値である
```

問【24】【円の描画】

高さ 600、幅 800、色が青色（RGB で (255,0,0)）の画像オブジェクトを定義し、中心に半径 10 の黄色の縁を描画するプログラム `ex24.py` を作成せよ。

3 マウス入力処理

2.7.2 矩形、線分の描画

矩形の描画には `cv2.rectangle()` 関数、線分の描画には `cv2.line()` 関数を用いる。

```
# 矩形の描画
cv2.rectangle(img, (x1, y1), (x2, y2), (b,g,r), weight)
# パラメータ：矩形の左上頂点の座標 (x1,y1)、右上頂点の座標 (x2,y2)、線の色、
# 線の太さ weight (負数の場合は塗りつぶし)

# 線分の描画
cv2.line(img, (x1,y1), (x2,y2), (b,g,r), weight)
# パラメータ：始点の座標 (x1,y1)、終点の座標 (x2,y2)、線の色、線の太さ
```

3 マウス入力処理

ウィンド上でマウス操作を行うとイベント（割り込み）が発生し、予め登録しておいたコールバック関数が呼び出される。コールバック関数とは、関数などを呼び出す際に引数で引き渡される別の関数のこと。呼び出し側の用意した関数などを呼び出し先のコードが「呼び出し返す」（callback）するので、コールバック関数と呼ばれる。

[コールバック関数の登録]

openCV のマウスイベントに対するコールバック関数の登録は以下のようにおこなう。

```
cv2.setMouseCallback("name", function_name, param)
```

name： マウスイベントを受け付けるウィンド名

function_name： コールバック関数名

param： コールバック関数に渡すパラメータ

[コールバック関数の定義]

コールバック関数の定義を以下に示す。

```
def onMouse(event, x, y, flags, param):
```

以下、コールバック関数の引数を説明する。

event： マウスイベントの種類、イベント一覧を表 3.1 に示す

表 3.1 マウスイベント一覧

イベント	発生するタイミング
cv2.EVENT_MOUSEMOVE	マウスを移動したとき, またはボタンを離した直後
cv2.EVENT_LBUTTONDOWN	左ボタンを押下したとき
cv2.EVENT_RBUTTONDOWN	右ボタンを押下したとき
cv2.EVENT_MBUTTONDOWN	中ボタンを押下したとき
cv2.EVENT_LBUTTONUP	左ボタンを離したとき
cv2.EVENT_RBUTTONUP	右ボタンを離したとき
cv2.EVENT_MBUTTONUP	中ボタンを離したとき
cv2.EVENT_LBUTTONDBLCLK	左ボタンがダブルクリックされたとき
cv2.EVENT_RBUTTONDBLCLK	右ボタンがダブルクリックされたとき
cv2.EVENT_MBUTTONDBLCLK	中ボタンがダブルクリックされたとき

x,y: イベントが発生したときのマウスポインタのウィンド上の座標

flags: マウスイベントが発生したときに同時に押されたキー、フラグ一覧を表 3.2 に示す

flags にはこれらのフラグの論理和がセットされる

表 3.2 フラグ一覧

フラグ名	フラグが立つ条件
cv2.EVENT_FLAG_LBUTTON	CTRL, SHIFT, ALT いずれも押さずに, 左ボタンを押下したとき
cv2.EVENT_FLAG_RBUTTON	CTRL, SHIFT, ALT いずれも押さずに, 右ボタンを押下したとき
cv2.EVENT_FLAG_MBUTTON	CTRL, SHIFT, ALT いずれも押さずに, 中ボタンを押下したとき
cv2.EVENT_FLAG_CTRLKEY	CTRL を押して, ボタンを離したときと離した直後
cv2.EVENT_FLAG_SHIFTKEY	SHIFT を押して, ボタンを離したときと離した直後
cv2.EVENT_FLAG_ALTKEY	ALT を押して, ボタンを離したときと離した直後

param: コールバック関数の登録時に設定したパラメータ

マウスが左クリックされた場合に中心に円を表示するプログラムを示す。

```

1 import numpy as np
2 import cv2
3
4 def onMouse(event, x, y, flags, param):
5     if event == cv2.EVENT_LBUTTONDOWN:
6         h,w = param[1].shape[:2]
7         cv2.circle(param[1],(int(w).int(h)),(255,0,0),20,-1)
8         cv2.imshow(param[0],param[1])
9
10 def main():
11     img = np.full((600,800,3),(0,255,255),dtype=np.uint8)
12     cv2.imshow("Img", img)
13
14     param = ('Img', img)
15     cv2.setMouseCallback("Img", onMouse, param)
16
17     cv2.waitKey(0)
18     cv2.destroyAllWindows()
19
20 if __name__ == '__main__':

```

問【25】【マウスイベント】

上記プログラムを修正し、マウスを右クリックした位置に円を描画するプログラム ex25.py を作成せよ

4 画像の平滑化

4.1 平滑化处理

画像には一般にノイズが含まれていることが多い。ノイズはランダムな細かい画素値の変動であり、画像処理の妨げになる。よって、物体検知などの画像処理を行う場合、画像に含まれているノイズを除去するために平滑化处理を行うことが多い。平滑化处理とは、図 4.1 において、真中の青い部分の画素値を、周りの画素値の平均値とするなどして求める処理で、画像を少しぼやかすことになる。

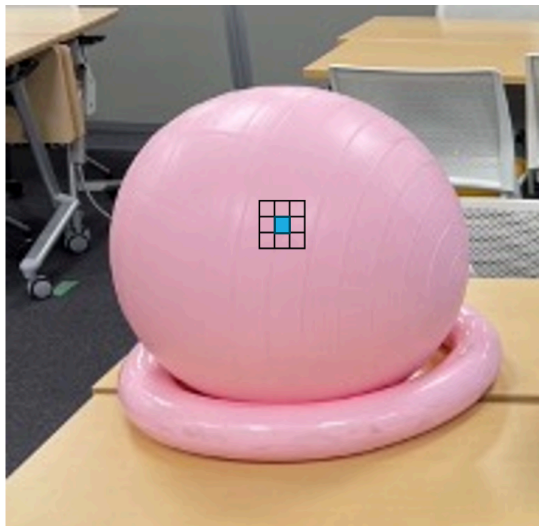


図 4.1 平滑化处理

どのように平均値を取るかを平滑化フィルタと呼ぶ。

平滑化フィルタには以下のフィルタがある。

- 移動平均フィルタ
- ガウシアンフィルタ
- メディアンフィルタ
- バイラテラルフィルタ

4.2 移動平均フィルタ

移動平均フィルタは注目画素とその周辺画素の値を同じ重みで平均を取り、注目画素の値とするフィルタである。どの範囲まで考えるかをカーネルと呼ぶ。カーネル3の場合のフィルタを図4.2に、カーネル5の場合のフィルタを図4.3に示す。

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

図 4.2 カーネル3のフィルタ

1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25

図 4.3 カーネル5のフィルタ

openCV で移動平均フィルタによる平滑化を行うには `cv2.blur()` を用いる。

```
img_blur = cv2.blur(img, ksize)
```

`img_blur` : 平滑化処理後の画像データ

`img` : 平滑化前の画像データ

`ksize` : カーネルサイズ、x 方向、y 方向の値をタプルで示す (例: (3, 3) など)

問【26】【移動平均フィルタ】

ball の画像を移動平均により平滑化するプログラム `ex26.py` を作成せよ。カーネルサイズを (5,5) とせよ。

4.2.1 ガウシアンフィルタ

ガウシアンフィルタでは注目画素に近いほど重みを大きくするフィルタである。重みは以下のガウス分布関数による。

$$f(x, y) = \frac{1}{2 \pi \sigma^2} \exp\left(-\frac{x^2 + y^2}{2 \sigma^2}\right)$$

ここで σ は標準偏差であり、 σ の値が大きいほどカーネルサイズがおおきいことに相当する。ガウシアンフィルタは移動平均フィルタに比較すると高周波のノイズ除去は効果がある。図 4.4 にカーネルサイズが 3 の場合のガウシアンフィルタを示す。

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

図 4.4 カーネル 3 のガウシアンフィルタ

openCV でガウシアンフィルタによる平滑化を行うには `cv2.GaussianBlur()` を用いる。

```
img_gauss = cv2.GaussianBlur(img, ksize, sigmaX, sigmaY)
```

`img_gauss` : 平滑化処理後の画像データ

`img` : 平滑化前の画像データ

`ksize` : カーネルサイズ、x 方向、y 方向の値をタプルで示す（例：(3,3) など）
奇数である必要がある

`sigmaX` : x 方向の標準偏差の値、0 とするとカーネルサイズから自動的に計算される

`sigmaY`: y 方向の標準偏差の値、省略すると `sigmaX` と同じ値になる

問【27】【ガウシアンフィルタ】

ball 画像をガウシアンフィルタにより平滑化するプログラム `ex27.py` を作成せよ。
カーネルサイズを (5,5)、`sigmaX` を 0 とせよ。

5 色認識

5.1 HSV 色空間

`cv2.imread()` 関数で画像を読み込むと、通常は RGB 情報として読み込まれる（実際は BGR の順）。RGB 色空間は R（レッド）、G（グリーン）、B（ブルー）の光の三原色を混ぜ合わせて色を表現する方法であり、OpenCV ではそれぞれ 0~255（8 ビット）の値を持つ。(0,0,0) は黒、(255,255,255) は白である。

RGB 色空間は RGB の混ぜ合わせ具合で色が決まるため、色認識が行いにくい。例えば、赤っぽい色の RGB 値がどうなるかは直感的には判断できない。そこで、色認識を行う場合には、色を直感的に認識しやすい HSV 空間に変換することが多い。

HSV 色空間とは、「色相 (Hue)」 「彩度 (Saturation)」 「明度 (Value)」 の 3 つの組み合わせで色を表現する手法である。

色相 (H) 色合い。赤っぽい、青っぽいといった色のおおまかな違いのことで、赤なら 0 度、黄色なら 60 度といったように角度で色合いが決まる。基本的には 0~360 の値であるが、OpenCV では 0~180 の範囲である。

彩度 (S) 色の鮮やかさ。色相が同じ場合でも、彩度が高ければ鮮やかに見え、低ければグレーに見える。彩度がゼロの場合は無彩色（黒、グレー、白）である。

明度 (V) 色の明るさ。高いほど明るい色になる。

色認識を行う場合には色相 (H) の値により行うことができる。

図 5.1 に三角錐による HSV 視覚化モデルを示す (<https://ja.wikipedia.org/wiki/HSV> 色空間)

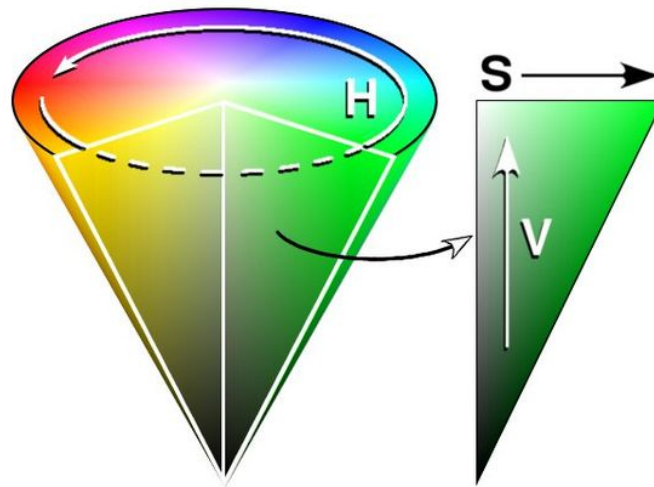


図 5.1 三角錐による HSV の視覚化モデル

色相（H）は三角錐の円周に沿って変化する。彩度（S）はその円錐の中心からの距離、明度（V）は円錐の頂点からの距離で表される。

BGR から HSV に変換するには `cv2.cvtColor()` を用いる。

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

hsv : HSV 色データ

img : BGR 色データ

OpenCV での HSV 色指定を表 5.1 に示す。

表 5.1 HSV の色指定の範囲

HSV	値の範囲
H（色相）	0～179
S（彩度）	0～255
V（明度）	0～255

赤、緑、青に対する openCV での HSV のおおよその値に範囲を表 5.2 に示す。

表 5.2 赤、緑、青に対する HSV 値

HSV	赤	緑	青
H (色相)	0～30、150～179	30～90	90～150
S (彩度)	64～255		
V (明度)	0～255		

問【28】【HSV 値】

ball.jpg を表示し、画面をクリックすると、その部分の色情報の値（RGB および HSV の値）を表示するプログラム ex28.py を作成せよ。

5.2 色によるマスク画像の作成

画像からある範囲の色の部分を取り出すには、色を指定して、その色の部分を白、他の色の部分を黒とするマスク画像を作成するとよい。

マスク画像（二値化画像）を作成するには cv2.inRange() を用いる。

```
mask = cv2.inRange(img, lower, upper)
```

mask：マスク画像（二値化画像）

img：元の画像データ

lower, upper：元の画像の 3 次元成分をタプルで指定、
lower と upper の間の値の場合に 255（白）、
それ以外の場合に 0（黒）となる

cv2.inRange() 関数のパラメータである lower と upper で色の範囲を指定する。その範囲の色の部分が白、その他の色の範囲が黒となる。

問【29】【色認識によるマスク画像の作成】

ball 写真の pink 色のボールの HSV での色の範囲を (165,32,0)~(179,255,255) として、マスク画像を作成するプログラム ex29.py を作成せよ。手順は以下。

- (1) ball 写真を読み込む
- (2) 画像を平滑化する
- (3) 平滑化した画像を HSV に変換する
- (4) cv2.inRange() 関数でマスク画像を作成し、表示する



図 5.2 元の ball の写真



図 5.3 pink で抽出

6 輪郭抽出

画像から物体の輪郭を抽出するには抽出したい物体を白としたマスク画像を作成し、そのマスク画像に cv2.findContours() 関数を用いる。cv2.findContours() では、画像の白の外側の輪郭を取得する。

```
contours, hierachy = cv2.findContours(mask, mode, method)
```

countours : 輪郭座標
hierachy : 輪郭の階層情報
mask : マスク画像 (二値化画像)
mode : 輪郭構造の取得方法

method：輪郭座標の取得情報

mode パラメータは白の輪郭の内側に更に白の輪郭がある場合に、輪郭情報をどのように取得するかを指定する。輪郭情報は戻り値である hierarchy に格納される。ここでは一番外側の輪郭を取得する場合を想定し、輪郭情報の詳細は省略する。この場合には mode パラメータに cv2.RETR_EXTERNAL を指定する。

表 6.1 に輪郭構造の取得方法を示す。

表 6.1 輪郭構造の取得方法

mode	輪郭構造の取得方法
cv2.RETR_EXTERNAL	一番外側の輪郭のみを取得する
cv2.RETR_LIST	全ての輪郭を同じ階層として取得する
cv2.RETR_CCOMP	白の輪郭の内側の輪郭の情報を取得する
cv2.RETR_TREE	全ての輪郭の階層構造を取得する

method パラメータには輪郭を構成する点の座標の取得方法を示す。

表 6.2 に輪郭座標の主な取得方法を示す。

表 6.2 輪郭座標の取得方法

mode	輪郭座標の取得方法
cv2.CHAIN_APPROX_NONE	輪郭上の全ての点の座標を取得する
cv2.CHAIN_APPROX_SIMPLE	輪郭の直線の部分の端点の座標を取得する

cv2.CHAIN_APPROX_NONE は全ての点の座標を取得するので多くのメモリを消費する。よって、通常は cv2.CHAIN_APPROX_SIMPLE を指定するとよい。

戻り値の contours は各輪郭を表すデータのリストとなっている。ここでは輪郭を表すデータの構造は省略する。

輪郭を描画するには cv2.drawContours() 関数を用いる。

```
cv2.drawContours(img, contours, index, color, thickness)
```

img：画像データ
contours：輪郭情報

`index`: `contours` のインデックス、-1 を指定すると全輪郭を描画する
`color`: 描画する線分の色を GBR のタプルで指定
`thickness`: 描画する線分の太さ

問【30】【輪郭抽出】

問【29】で求めたマスク画像から輪郭を抽出して描画するプログラム `ex30.py` を作成せよ。手順は以下。

- (1) `ball` 写真を読み込む
- (2) 画像を平滑化する
- (3) 平滑化した画像を HSV に変換する
- (4) `cv2.inRange()` 関数でマスク画像を作成する（ここまでは問【29】）
- (5) マスク画像から `cv2.findContours()` 関数を用いて輪郭を抽出する
- (6) 抽出した輪郭を元の画像に描画する

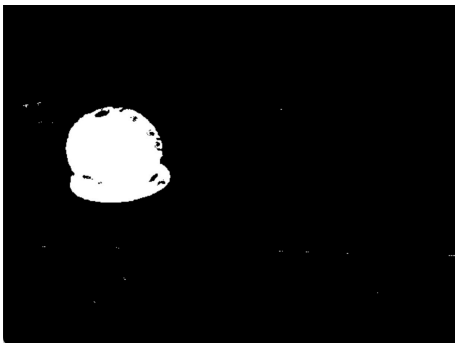


図 6.1 pink で抽出

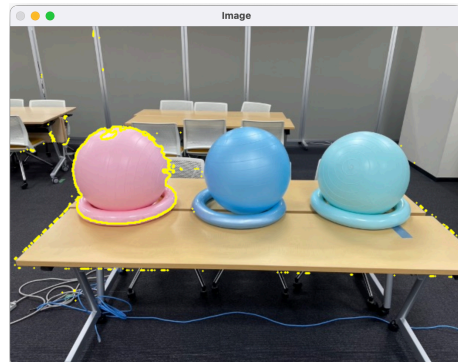


図 6.2 元の ball の写真

輪郭を表すデータを使って輪郭の各種特徴情報を取得することができる。

表 6.3 に輪郭を表すデータを `cnt` とした場合の特徴情報取得方法を示す（`contours` は `cnt` のリストである）。

表 6.3 輪郭特徴情報の取得方法

特徴情報	輪郭特徴情報の取得方法	備考
面積	<code>cv2.contourArea(cnt)</code>	
周囲の長さ	<code>cv2.arcLength(cnt, True)</code>	
輪郭に外接する矩形	<code>x,y,w,h = cv2.boundingRect(cnt)</code>	戻り値：矩形の左上の x,y 座標、矩形の幅、高さ
輪郭の最小外接円	<code>(x,y),r = cv2.minEnclosingCircle(cnt)</code>	戻り値：外接円の中心座標 (x,y) 、半径 r

問【31】【外接矩形の描画】

ball の写真から指定した pink 色の ball を抽出し、外接矩形を黄色で描画するプログラム `ex31.py` を作成せよ。

[ヒント] 抽出した輪郭から ball を選択するには、面積が一番大きい輪郭を選べば良い

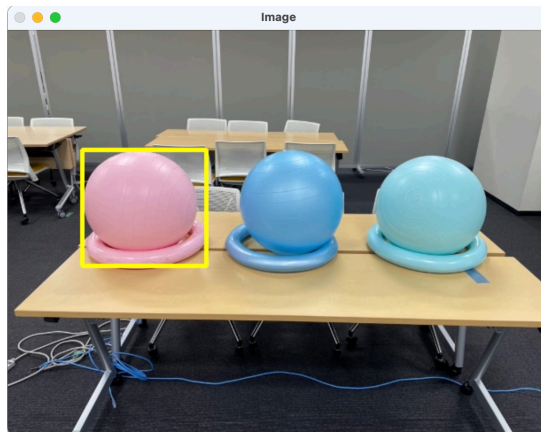


図 6.3 ball の写真から pink の ball の外接矩形を描画

【cv2.rectangle() 関数の引数について】

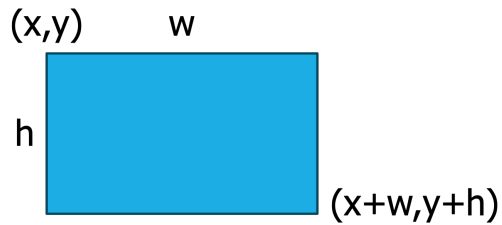


図 6.4 矩形の座標

これまで、矩形を描画する場合、cv2.rectangle() 関数の引数は以下としてきた。

```
矩形の左上の座標 (x,y)、矩形の幅、高さ (w,h)  
cv2.rectangle(img,(x,y),(x+w,y+h),color,thickness)
```

しかし、以下のパラメータでも描画できる。

```
矩形の左上の座標 (x,y)、矩形の幅、高さ (w,h)  
cv2.rectangle(img,(x,y,w,h),color,thickness)  
rect = (x,y,w,h)  
cv2.rectangle(img,rect,color,thickness)
```